



`alittlebyte.swe@gmail.com`

Specifica Tecnica

Gruppo	aLittleByte (gruppo 19)
Versione	1.0.0
Redattori	–
Verificatori	–
Destinatari	Prof. Tullio Vardanega, Prof. Riccardo Cardin
Data	–

Registro delle Modifiche

Versione	Data	Descrizione	Redattore	Verificatore	Approvatore
----------	------	-------------	-----------	--------------	-------------

Indice

1	Introduzione	4
1.1	Contenuto del documento	4
1.2	Struttura del documento	4
1.3	Glossario	4
1.4	Riferimenti	4
1.4.1	Riferimenti normativi	4
1.4.2	Riferimenti informativi	5
2	Tecnologie	5
2.1	Linguaggi di programmazione	5
2.2	Framework	6
2.3	Librerie	6
2.4	Contratto API	7
2.5	Ambiente locale e provisioning	8
2.6	Edge e web server	9
2.7	Servizi cloud AWS	9
2.8	Tecnologie per la persistenza dei dati	11
2.9	Osservabilità	11
2.10	CI e quality gate	12
3	API	14
3.1	State API	14
3.2	Communications API	14
3.3	Documents API	16
4	Architettura del Sistema	17
4.1	Contesto di Sistema	17
4.2	Architettura Applicativa	17
4.3	Architettura Logica Interna	17
4.3.1	Frontend: Interfaccia Utente	17
4.3.2	Backend: Architettura Esagonale	18
4.3.2.1	Adapter primari	18
4.3.2.2	Dominio e applicazione	18
4.3.2.3	Adapter secondari	18
4.3.2.4	Esempio: il dominio Documents	18
4.4	Dettaglio dei Componenti	18
4.4.0.1	Documents	19
4.4.0.2	Communications	19
4.5	Architettura di Deployment	20
4.6	Containerizzazione con Docker	20
4.7	Stile architetturale	20

1 Introduzione

1.1 Contenuto del documento

Il presente documento costituisce la Specifica Tecnica del sistema sviluppato dal gruppo *aLittleByte* per il proponente Eggon S.r.l. nell'ambito del progetto Nexum.

Al suo interno vengono illustrate le tecnologie selezionate, il contratto delle API REST esposte dal backend e le scelte architetturali adottate nella realizzazione del prodotto. Il documento ha lo scopo di fornire una visione organica e dettagliata della struttura del sistema, delle sue componenti e delle interazioni tra esse.

Deve essere inteso come riferimento tecnico per le attività di progettazione e codifica, garantendo la coerenza tra le decisioni implementative adottate e quanto emerso nella fase di Proof of Concept.

1.2 Struttura del documento

La struttura del presente documento è articolata nelle seguenti sezioni principali:

- **Sezione 1 - Introduzione:** Presenta il contenuto e gli obiettivi del documento, il glossario dei termini e i riferimenti normativi e informativi adottati.
- **Sezione 2 - Tecnologie:** Descrive lo stack tecnologico adottato, suddiviso per categoria: linguaggi, framework, librerie, ambiente locale, edge, servizi cloud AWS, persistenza dei dati, osservabilità e CI/quality gate.
- **Sezione 3 - API:** Descrive il contratto REST esposto dal backend, organizzato per area funzionale (stato applicativo, AI Assistant Generativo, AI Co-Pilot).
- **Sezione 4 - Architettura del Sistema:** Descrive il contesto di sistema, l'architettura applicativa e logica interna del backend, il dettaglio dei componenti, il deployment, la containerizzazione e lo stile architetturale complessivo.

1.3 Glossario

Al fine di evitare ambiguità e garantire una comprensione uniforme della terminologia utilizzata, i termini tecnici, gli acronimi e le parole con un significato specifico nel contesto del progetto sono contrassegnati nel testo con una “G” ad apice (es. parola^G). La loro definizione completa è consultabile nel documento esterno *Glossario*.

1.4 Riferimenti

1.4.1 Riferimenti normativi

- **Capitolato d'appalto C5 - NEXUM (Eggon S.r.l.):**
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C5.pdf>
Data Ultimo accesso: 09/07/2026

- **Norme di Progetto v1.0.0:**

Documento interno del gruppo *aLittleByte* che definisce le regole, i ruoli e le procedure operative adottate nel progetto.

<https://alittlebyte-19.github.io/Documentazione/RTB/Norme%20di%20Progetto/Norme%20di%20Progetto.pdf>

Data Ultimo accesso: 09/07/2026

- **Regolamento del Progetto Didattico:**

Anno accademico 2025/2026, Corso di Ingegneria del Software, Università di Padova.

<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/PD1.pdf>

Data Ultimo accesso: 09/07/2026

1.4.2 Riferimenti informativi

- **Glossario v2.0.0:**

Documento interno del gruppo *aLittleByte* contenente le definizioni dei termini tecnici e degli acronimi utilizzati nella documentazione.

<https://alittlebyte-19.github.io/Documentazione/RTB/Glossario/Glossario.pdf>

Data Ultimo accesso: 09/07/2026

- **Analisi dei Requisiti v2.0.0:**

Documento interno del gruppo *aLittleByte* contenente la descrizione dei requisiti funzionali e non funzionali del sistema.

<https://alittlebyte-19.github.io/Documentazione/RTB/Analisi%20dei%20Requisiti/Analisi%20dei%20Requisiti.pdf>

Data Ultimo accesso: 09/07/2026

2 Tecnologie

Il presente capitolo illustra lo stack tecnologico adottato per la progettazione e la realizzazione del sistema Nexum. Le scelte implementative sono state guidate dai requisiti di vincolo definiti nell'Analisi dei Requisiti e dalle indicazioni del proponente Eggon S.r.l. Lo stack è organizzato attorno a cinque macro-aree principali: il runtime applicativo (backend Laravel su PHP e frontend Angular in TypeScript), l'infrastruttura cloud AWS per l'elaborazione intelligente dei documenti e la gestione asincrona dei job, la persistenza dei dati, lo stack di osservabilità e la pipeline di integrazione continua con quality gate automatici. L'ambiente locale replica fedelmente la configurazione di produzione attraverso la containerizzazione con Docker e l'emulazione dei servizi cloud con LocalStack, garantendo riproducibilità e coerenza tra i diversi ambienti di sviluppo e produzione.

2.1 Linguaggi di programmazione

Nome	Versione	Descrizione
PHP	8.4	Linguaggio di scripting server-side utilizzato per la realizzazione del backend tramite il framework Laravel. La versione 8.4 introduce miglioramenti significativi alle performance e nuove funzionalità del linguaggio.
TypeScript	5.9	Superset tipizzato di JavaScript sviluppato da Microsoft, impiegato per la realizzazione del frontend Angular. La tipizzazione statica riduce gli errori a tempo di compilazione e migliora la manutenibilità del codice.
SQL	SQL:2023	Linguaggio dichiarativo per la definizione e la manipolazione dei dati nel database relazionale PostgreSQL, utilizzato per la gestione della persistenza strutturata del sistema.

Tabella 2: Linguaggi di programmazione

2.2 Framework

Nome	Versione	Descrizione
Laravel	12	Framework PHP open source per lo sviluppo di applicazioni web server-side. Fornisce un'architettura MVC, un sistema di routing, un ORM (Eloquent) e strumenti integrati per la gestione dei job asincroni. È stato scelto per la maturità del suo ecosistema rispetto ad alternative come Symfony.
Angular	21	Framework open source sviluppato da Google per la creazione di Single Page Application (SPA). Utilizza TypeScript e adotta un'architettura basata su componenti con supporto nativo per dependency injection, routing, reactive forms e signals. È stato preferito per la struttura integrata e opinionata che garantisce coerenza e manutenibilità nel tempo.

Tabella 3: Framework

2.3 Librerie

Nome	Versione	Ambito	Descrizione
RxJS	7.8	Frontend	Libreria per la programmazione reattiva basata su Observable. Utilizzata per la gestione dello stato asincrono e degli stream di eventi all'interno della SPA Angular, in combinazione con i signals nativi di Angular.

Nome	Versione	Ambito	Descrizione
@lucide/angular	1.21	Frontend	Libreria di icone open source integrata nel frontend Angular. Fornisce un set di icone SVG consistente e accessibile per l'interfaccia utente.
AWS SDK for PHP	3.380	Backend	SDK ufficiale AWS per PHP. Utilizzata dal backend Laravel per invocare Bedrock, Textract, S3, SQS, Step Functions, SSM Parameter Store e Secrets Manager tramite un'unica libreria client.
Flysystem S3 (league/flysystem-aws-s3-v3)	3.32	Backend	Adapter S3 per il filesystem astratto di Laravel. Consente di trattare i bucket S3 (o LocalStack) come un disco Laravel standard, cambiando solo la configurazione tra ambienti.
Opis JSON Schema	2.6	Backend	Libreria per la validazione di payload JSON contro uno schema. Utilizzata per validare la struttura dei dati restituiti dai modelli Bedrock durante l'estrazione dei documenti.
FPDI	2.6	Backend	Libreria PHP per l'importazione di pagine da documenti PDF esistenti. Utilizzata nella pipeline documentale per estrarre pagine specifiche da PDF massivi prima dello split.
FPDF	1.8	Backend	Libreria PHP per la generazione di documenti PDF. Utilizzata in combinazione con FPDI per produrre i sotto-documenti PDF per range di pagine a partire dai file originali.

Tabella 4: Librerie

2.4 Contratto API

Il frontend non conosce gli endpoint del backend in modo statico: il contratto OpenAPI 3.1 costituisce la fonte di verità condivisa da cui viene generato automaticamente il client TypeScript tipizzato, mantenendo frontend e backend allineati durante l'evoluzione del sistema.

Nome	Versione	Descrizione
OpenAPI	3.1	Specifica standard per la descrizione delle API REST. Costituisce il contratto condiviso tra frontend e backend e rappresenta la fonte di verità da cui vengono derivati client e documentazione.
Orval	8.18	Strumento di code generation che, a partire dalla specifica OpenAPI 3.1, genera automaticamente il client TypeScript tipizzato per il frontend Angular, eliminando la scrittura manuale delle chiamate HTTP.

Nome	Versione	Descrizione
Redocly CLI	latest	Strumento per la validazione e il linting del contratto OpenAPI, eseguito in pipeline sempre nell'ultima versione disponibile. Verifica la conformità della specifica agli standard e individua inconsistenze prima che vengano propagate al codice generato.

Tabella 5: Strumenti per il contratto API

2.5 Ambiente locale e provisioning

L'ambiente locale è progettato in modo che ogni membro del team lavori sulla stessa identica configurazione, vicina a quella di produzione, senza dover creare account cloud o installare servizi manualmente. I servizi AWS vengono emulati in locale tramite LocalStack, mentre l'infrastruttura è descritta come codice con Terraform, rendendola versionata e ricreabile.

Nome	Versione	Descrizione
Docker	—	Piattaforma di containerizzazione utilizzata per costruire le immagini dell'applicazione e del web server. Garantisce che l'ambiente di esecuzione sia identico per tutti i membri del team e coerente con la produzione.
Docker Compose	—	Strumento per l'orchestrazione dei container locali. Avvia l'intera infrastruttura (applicazione, database, servizi di supporto) su reti separate con un singolo comando.
Make	—	Automazione dei comandi ricorrenti tramite Makefile. Garantisce che le procedure di setup, avvio e build siano identiche per tutti i membri del team.
Composer	—	Gestore delle dipendenze PHP. Gestisce le librerie del backend e le relative versioni, garantendo riproducibilità delle build.
Node.js	22	Runtime JavaScript utilizzato dagli strumenti di build e sviluppo del frontend (Angular CLI, npm, Orval, ESLint, Jest). Eseguito in un container dedicato, non richiede un'installazione locale sulla macchina dello sviluppatore.
LocalStack	4.5	Emulatore locale dei servizi AWS. Riproduce in locale S3, SQS, Step Functions, SSM, Secrets Manager e KMS, consentendo di testare il flusso completo senza dipendere da risorse cloud attive o credenziali reali.
Terraform	1.10.5	Strumento di Infrastructure as Code (IaC) per il provisioning dell'infrastruttura. Sostituisce gli script manuali con una descrizione dichiarativa e versionata, ricreabile a comando tramite <code>terraform apply</code> .

Nome	Versione	Descrizione
AWS SSM Parameter Store	—	Servizio AWS per la gestione della configurazione runtime non sensibile. I parametri vengono letti dall'applicazione al bootstrap, mantenendo la configurazione fuori dal codice sorgente.
AWS Secrets Manager	—	Servizio AWS per la gestione dei segreti applicativi (credenziali, chiavi API). Mantiene i dati sensibili separati dal codice e dalla configurazione.

Tabella 6: Strumenti per l'ambiente locale e il provisioning

2.6 Edge e web server

Una richiesta HTTPS attraversa diversi livelli prima di raggiungere il codice applicativo. Traefik opera come entry point TLS esterno; in locale un secondo Nginx (*edge-cdn*) emula il ruolo di una CDN, servendo la SPA Angular direttamente da S3/LocalStack e inoltrando le richieste `/api`, `/health` e `/ready` all'Nginx applicativo, che a sua volta fa da proxy verso PHP-FPM. In produzione il ruolo di CDN è coperto da AWS CloudFront, mentre l'Nginx applicativo resta un'immagine dedicata esclusivamente al proxy verso PHP-FPM.

Nome	Versione	Descrizione
Traefik	3.4	Reverse proxy posizionato davanti a tutti i servizi. Si occupa della terminazione TLS, dello smistamento delle richieste in base all'host e dell'esposizione delle metriche per il sistema di osservabilità.
Nginx (applicativo)	1.27	Web server utilizzato come proxy per le chiamate <code>/api</code> verso PHP-FPM tramite FastCGI. Immagine di produzione, buildata e scansionata, che non conosce LocalStack.
Nginx (edge-cdn)	1.29	Secondo Nginx, presente solo in locale, che emula il ruolo di una CDN/edge davanti a S3-LocalStack per servire gli asset statici della SPA Angular. In produzione questo ruolo è sostituito da AWS CloudFront.
PHP-FPM	8.4	Runtime PHP con processo FastCGI. Esegue il codice dell'applicazione Laravel. È stato preferito a FrankenPHP e Laravel Octane per un comportamento più prevedibile e una gestione dei processi più semplice in fase di PoC.

Tabella 7: Edge e web server

2.7 Servizi cloud AWS

Le funzionalità di intelligenza artificiale, l'elaborazione asincrona dei documenti e la protezione dei dati si basano su servizi gestiti AWS. L'adozione di servizi gestiti elimina la necessità di infrastrutture dedicate, garantendo scalabilità automatica e alta disponibilità. In locale tutti questi servizi vengono emulati da LocalStack.

Nome	Versione	Descrizione
Amazon Bedrock	—	Servizio AWS per l'accesso a modelli linguistici di grandi dimensioni (LLM). Di default utilizza Amazon Nova Lite per la generazione dei testi e Stability SD3.5 Large per le immagini di copertina nell'AI Assistant Generativo, oltre che per la classificazione e lo split automatico dei documenti nell'AI Co-Pilot.
Amazon Textract	—	Servizio AWS di riconoscimento ottico dei caratteri (OCR). Estrae testo e dati strutturati da documenti scansionati o immagini, leggendo i file direttamente da S3.
AWS Step Functions	—	Servizio di orchestrazione delle pipeline asincrone. Due state machine distinte coordinano rispettivamente la pipeline documentale (OCR, estrazione AI, persistenza risultati) e la pipeline delle comunicazioni (generazione testo/copertina), gestendo tentativi, timeout e stato tramite task token. È stato preferito a soluzioni self-hosted come Temporal o Camunda.
Amazon SQS	—	Servizio di code di messaggi gestite. Sono presenti due code separate, una per la pipeline documentale e una per quella delle comunicazioni, scelte per la loro durabilità e per la gestione automatica dei messaggi falliti. Un Worker dedicato per pipeline consuma i messaggi dalla propria coda e risponde all'orchestratore Step Functions con un token di callback.
Amazon SQS DLQ	—	Dead Letter Queue: una coda secondaria SQS per pipeline che raccoglie automaticamente i messaggi che hanno superato il numero massimo di tentativi (<code>maxReceiveCount</code>), consentendo l'analisi e il reindirizzamento dei task falliti.
Amazon S3	—	Servizio di object storage. Un bucket dedicato archivia i documenti originali, i sotto-documenti generati dalla pipeline e le copertine delle comunicazioni, con oggetti cifrati at-rest tramite KMS; un secondo bucket ospita gli asset statici della SPA Angular, serviti dall'edge locale o da CloudFront in produzione. L'accesso a entrambi è regolato da policy IAM.
AWS KMS	—	Servizio di gestione delle chiavi crittografiche. Utilizzato per la cifratura at-rest degli oggetti del bucket documentale, garantendo la protezione dei documenti sensibili.
AWS IAM	—	Servizio di gestione delle identità e degli accessi. Definisce i ruoli di esecuzione per Step Functions e le policy di accesso alle risorse, seguendo il principio del minimo privilegio.

Tabella 8: Servizi cloud AWS

2.8 Tecnologie per la persistenza dei dati

I dati del sistema hanno nature diverse e vengono gestiti con strumenti dedicati: PostgreSQL per la persistenza strutturata e transazionale, Redis per lo stato volatile ad accesso rapido.

Nome	Versione	Descrizione
PostgreSQL	16	Sistema di gestione di database relazionali open source, erogato come servizio gestito in cloud. Conserva i dati strutturati dell'applicazione (documenti, estratti, audit, stato dei task) e, grazie al supporto JSONB, gestisce anche payload semi-strutturati senza cambiare database. È stato preferito a MySQL e a soluzioni documentali come MongoDB per affidabilità e flessibilità.
Redis	7	Data store in-memory open source, erogato come servizio gestito in cloud. Gestisce la cache applicativa, le sessioni utente e il rate limiting, sempre con scadenza. Non viene utilizzato come coda della pipeline, ruolo affidato ad Amazon SQS.

Tabella 9: Tecnologie per la persistenza dei dati

2.9 Osservabilità

Il sistema è strumentato con OpenTelemetry, uno standard indipendente dal fornitore che consente di raccogliere metriche, tracce e log senza vincolarsi a prodotti specifici. I log applicativi vengono inviati via OTLP al Collector, mentre le metriche sono esposte dall'applicazione su un endpoint dedicato (`/internal/metrics`) e raccolte dal Collector tramite scraping; entrambi i segnali vengono poi smistati verso i rispettivi backend di storage, mentre Grafana li correla in cruscotti unificati.

Nome	Versione	Descrizione
OpenTelemetry (OTel)	—	Standard open source per la strumentazione delle applicazioni. Definisce il formato OTLP con cui log (e potenzialmente tracce) vengono inviati al Collector, consentendo di cambiare backend di osservabilità senza modificare il codice applicativo.
OTel Collector	0.153.0	Componente centrale che riceve i log via OTLP e le metriche tramite scraping (Nginx, Traefik e se stesso), smistandoli verso Prometheus (metriche), Tempo (tracce) e Loki (log).
Prometheus	3.9.0	Sistema di raccolta e storage delle metriche. Raccoglie le metriche esposte dai servizi tramite scrape e definisce le regole su cui Alertmanager genera gli avvisi.

Nome	Versione	Descrizione
Grafana Tempo	2.9.0	Backend per le tracce distribuite. Riceve le tracce dal Collector e consente la correlazione con metriche e log all'interno di Grafana.
Grafana Loki	3.3.2	Sistema di aggregazione dei log. Riceve i log dal Collector via OTLP e da Alloy, indicizzandoli per consentire ricerche contestuali.
Grafana Alloy	1.5.1	Agente per la raccolta dei log dai container Docker Compose. Invia i log di container a Loki tramite pipeline di raccolta.
Grafana	12.3.0	Piattaforma di visualizzazione. Correla metriche, tracce e log in cruscotti unificati, consentendo l'analisi contestuale degli eventi senza dover ricorrere a stack separati come ELK o Jaeger.
Alertmanager	0.29.0	Componente per la gestione e l'instradamento degli alert. Genera avvisi a partire da regole definite su Prometheus, legate a eventi concreti del dominio applicativo.

Tabella 10: Stack di osservabilità

2.10 CI e quality gate

Ogni modifica inviata al repository attiva la pipeline di integrazione continua su GitHub Actions, che ricostruisce le immagini e le pubblica su GHCR solo dopo che tutti i controlli automatici sono stati superati. I quality gate coprono stile del codice, analisi statica, test, sicurezza delle dipendenze e accessibilità.

Nome	Versione	Categoria	Descrizione
GitHub Actions	—	CI/CD	Orchestratore della pipeline CI. Attivato da push e pull request, esegue in parallelo le pipeline di backend, frontend e stack.
GHCR	—	CI/CD	GitHub Container Registry: registro in cui vengono pubblicate le immagini Docker solo dopo il superamento di tutti i quality gate.
Docker Hub	—	CI/CD	Utilizzato esclusivamente per il pull autenticato delle immagini base, al fine di evitare i limiti di rate anonimo. Non è il registro di pubblicazione delle immagini prodotte.

Nome	Versione	Categoria	Descrizione
Trivy	0.66.0	Sicurezza	Scanner di vulnerabilità per immagini Docker. Analizza le immagini costruite (vulnerabilità, secret, configurazione) prima della pubblicazione su GHCR.
npm audit	—	Sicurezza	Analisi delle vulnerabilità nelle dipendenze npm di produzione del frontend Angular.
Pint	1.29	Qualità backend	Formattatore e linter per il codice PHP, basato sulle convenzioni di Laravel.
PHPStan / Larastan	3.10	Qualità backend	Strumento di analisi statica per PHP con estensione specifica per Laravel. Individua errori di tipo e problemi di logica senza eseguire il codice.
Pest	4.7	Qualità backend	Framework di testing per PHP utilizzato per i test unitari e di integrazione del backend Laravel.
ESLint	9.39	Qualità frontend	Linter per il codice TypeScript della SPA Angular. Verifica la conformità alle regole di stile e individua potenziali errori.
Jest	30.4	Qualità frontend	Framework di testing JavaScript utilizzato per i test unitari del frontend Angular, in combinazione con TypeScript.
Redocly + Orval diff	8.18	Contratto API	Redocly (sempre ultima versione) valida il contratto OpenAPI e Orval rigenera il client tipizzato; un diff verifica che il client generato sia allineato con il contratto aggiornato.
terraform validate	1.10.5	IaC	Valida formato e correttezza sintattica/semantica della configurazione Terraform prima di qualsiasi applicazione.
axe	4.12	Accessibilità	Strumento di testing automatico per l'accessibilità (a11y) dell'interfaccia utente, integrato nella pipeline CI.
pally	9.1	Accessibilità	Strumento complementare ad axe per il testing dell'accessibilità. Esegue test a11y sulle pagine dell'applicazione.
Playwright	1.60	Accessibilità	Framework per l'automazione di un browser reale (Chromium). Utilizzato per eseguire gli audit di accessibilità con axe e Pally e per verificare che la SPA funzioni correttamente con la Content Security Policy applicata.

Nome	Versione	Categoria	Descrizione
------	----------	-----------	-------------

Tabella 11: Strumenti per CI e quality gate

3 API

Il sistema espone un contratto REST versionato (`/api/v1`), definito tramite la specifica OpenAPI 3.1 in `openapi/v1/alittlebyte-mvp-api.yaml` e consumato dalla SPA Angular tramite il client generato con Orval. Ogni richiesta è autenticata tramite l'header `X-Mvp-User-Id` (schema `MvpIdentity`), che veicola le identity claims iniettate dal confine aziendale o dal middleware locale della MVP.

Il rate limiting è granulare per gruppo di endpoint: 60 richieste/minuto di default, 20/minuto per le operazioni di generazione e scrittura più onerose (creazione bozze, rigenerazione, upload OCR, aggiornamento copertina, salvataggio configurazioni di prompt), 30/minuto per anteprima, export e valutazione, 120/minuto per la sola lettura della copertina già generata. Gli endpoint che restituiscono un PDF (anteprima ed export delle comunicazioni) supportano la cache HTTP tramite `ETag/If-None-Match`, evitando di rigenerare il documento se il client possiede già l'ultima versione.

L'avanzamento delle elaborazioni asincrone (generazione contenuti, pipeline documentale) è notificato alla SPA tramite endpoint di streaming Server-Sent Events (SSE), evitando il polling. Le liste di comunicazioni e documenti supportano filtri lato server e paginazione. Le API sono organizzate in tre aree funzionali, corrispondenti ai tag della specifica OpenAPI: lo stato applicativo, l'AI Assistant Generativo (comunicazioni) e l'AI Co-Pilot (documenti).

3.1 State API

Espone lo stato corrente dei due moduli applicativi, utilizzato dalla SPA per popolare le viste principali senza richieste multiple.

Endpoint	Metodo	Descrizione
<code>/api/v1/state</code>	GET	Restituisce in un'unica risposta le metriche e lo storico delle comunicazioni dell'AI Assistant Generativo, insieme alle metriche e ai sotto-documenti dell'AI Co-Pilot.

Tabella 12: State API

3.2 Communications API

Gestione del ciclo di vita completo di una comunicazione generata dall'AI Assistant Generativo: richiesta della bozza, revisione e modifica, rigenerazione, salvataggio o scarto, valutazione, esportazione e gestione della copertina. Include inoltre la gestione delle configurazioni di prompt salvate per il riuso.

Endpoint	Metodo	Descrizione
/api/v1/communications	GET	Lista delle bozze di comunicazione del tenant, con filtri opzionali (parola chiave nel prompt, tono, stile, data) e paginazione; esclude le bozze scartate.
/api/v1/communications	POST	Richiede una bozza di comunicazione a partire da prompt, tono e stile; avvia in modo asincrono la pipeline delle comunicazioni e restituisce l'identificativo della comunicazione e l'URL di streaming.
/api/v1/communications/{communication}	PUT	Aggiorna titolo e corpo di una bozza di comunicazione.
/api/v1/communications/{communication}	DELETE	Elimina definitivamente una comunicazione dallo storico.
/api/v1/communications/{communication}/regenerate	POST	Rigenera la bozza di comunicazione mantenendo o aggiornando i parametri di prompt, tono e stile.
/api/v1/communications/{communication}/save	POST	Salva la bozza nello storico, lasciandola comunque modificabile e rigenerabile.
/api/v1/communications/{communication}/discard	POST	Scarta la bozza di comunicazione.
/api/v1/communications/{communication}/favorite	POST	Aggiunge la comunicazione ai preferiti.
/api/v1/communications/{communication}/favorite	DELETE	Rimuove la comunicazione dai preferiti.
/api/v1/communications/{communication}/rating	POST	Assegna una valutazione da 1 a 5 stelle alla bozza generata; valutazioni successive per la stessa generazione vengono rifiutate.
/api/v1/communications/{communication}/stream	GET	Endpoint SSE che notifica alla SPA l'avanzamento della generazione (testo e copertina) per una comunicazione specifica.
/api/v1/communications/{communication}/cover-image	GET	Download dell'immagine di copertina generata per la comunicazione (PNG, JPEG o WebP).
/api/v1/communications/{communication}/cover-image	POST	Sostituisce manualmente l'immagine di copertina generata dall'AI, caricando un file.
/api/v1/communications/{communication}/cover-image	DELETE	Rimuove l'immagine di copertina associata alla comunicazione.
/api/v1/communications/{communication}/preview	GET	Anteprima del PDF impaginato della comunicazione, con supporto ETag per evitare rigenerazioni non necessarie.

Endpoint	Metodo	Descrizione
/api/v1/communications/{communication}/export	GET	Download del PDF finale della comunicazione come allegato, con lo stesso supporto ETag dell'anteprima.
/api/v1/prompt-configurations	POST	Salva la configurazione corrente di prompt (tono, stile, testo) per un riutilizzo successivo.
/api/v1/prompt-configurations/{promptConfiguration}	DELETE	Elimina definitivamente una configurazione di prompt salvata.

Tabella 13: Communications API

3.3 Documents API

Gestione del ciclo di vita dei documenti caricati dall'AI Co-Pilot, della relativa pipeline di elaborazione asincrona (OCR, estrazione, revisione) e della composizione del messaggio di invio verso il destinatario individuato.

Endpoint	Metodo	Descrizione
/api/v1/documents	GET	Lista dei sotto-documenti del tenant, con filtri opzionali (ricerca su nome/cognome/azienda, stato di invio, soglia e criterio di confidenza, mese/anno) e paginazione.
/api/v1/documents/ocr	POST	Carica un documento e avvia in modo asincrono la pipeline documentale (OCR tramite Textract, estrazione e classificazione tramite Bedrock); restituisce l'URL di streaming per seguirne l'avanzamento.
/api/v1/documents/{originalDocument}/stream	GET	Endpoint SSE che notifica alla SPA l'avanzamento dell'elaborazione per un documento originale specifico.
/api/v1/documents/{subDocument}	DELETE	Elimina un sotto-documento già elaborato dalla pipeline.
/api/v1/documents/{subDocument}/extracted-data	PUT	Corregge i dati estratti automaticamente per un sotto-documento (dati anagrafici, tipo documento, data, descrizione, punteggio di confidenza).
/api/v1/documents/{subDocument}/review	POST	Segna un sotto-documento come validato manualmente, ad esempio a valle di una revisione human-in-the-loop per i casi a bassa confidenza.
/api/v1/documents/{subDocument}/preview	GET	Restituisce l'anteprima in formato PDF del sotto-documento.

Endpoint	Metodo	Descrizione
/api/v1/documents/{subDocument}/send-preview	GET	Anteprima PDF del messaggio di invio precompilato per il sotto-documento.
/api/v1/documents/{subDocument}/send-export	GET	Download del messaggio di invio precompilato come PDF allegato.
/api/v1/documents/{subDocument}/send-message	PUT	Corregge i campi del messaggio di invio precompilato per il sotto-documento.

Tabella 14: Documents API

4 Architettura del Sistema

Questo capitolo descrive l'architettura del sistema, dal generale al particolare: contesto, architettura applicativa, architettura logica interna del backend, dettaglio dei componenti, deployment, containerizzazione e stile architetturale complessivo.

4.1 Contesto di Sistema

L'utente finale interagisce con la piattaforma tramite browser, via HTTPS. Il sistema delega l'elaborazione intelligente dei contenuti a due servizi AWS esterni: l'AI Co-Pilot invia i documenti caricati ad AWS Textract per il riconoscimento ottico dei caratteri; l'AI Assistant Generativo invia i prompt ad AWS Bedrock per la generazione di testi e immagini. Le integrazioni con AWS avvengono esclusivamente lato backend; il browser comunica solo con il sistema.

4.2 Architettura Applicativa

Il sistema si compone dei seguenti container principali. Il frontend è una Single Page Application in Angular 21. Il backend è un'API in Laravel 12 su PHP 8.4, servita da PHP-FPM e Nginx, che espone esclusivamente un contratto REST in formato JSON, senza rendering lato server. Le operazioni pesanti — OCR e invocazione dei modelli generativi — sono delegate in modo asincrono ad AWS Step Functions e Amazon SQS, consumate da un processo Worker separato dall'API. La persistenza è affidata a PostgreSQL per i dati applicativi e a Redis per cache, sessioni e rate limiting; nessuno dei due è utilizzato come coda, ruolo riservato a SQS.

4.3 Architettura Logica Interna

Il frontend è dedicato all'interfaccia utente. Il backend è organizzato per dominio applicativo secondo i principi dell'architettura esagonale (*ports & adapters*).

4.3.1 Frontend: Interfaccia Utente

Sviluppato in Angular come Single Page Application, gestisce l'interfaccia e l'interazione con l'utente finale.

- Gestisce gli input dell'utente (caricamento documenti, richieste di generazione, revisione dei dati estratti).
- Comunica con il backend tramite chiamate API RESTful, generate a partire dal contratto OpenAPI condiviso.
- Riceve l'avanzamento delle elaborazioni asincrone tramite Server-Sent Events, senza polling.

4.3.2 Backend: Architettura Esagonale

Il backend adotta il pattern *ports & adapters*, applicato ai due domini di prodotto: **Documents** (AI Co-Pilot) e **Communications** (AI Assistant Generativo). La logica di business è disaccoppiata dai dettagli tecnici — framework, database, SDK esterni — per garantire manutenibilità, testabilità e sostituibilità dei singoli componenti. Il backend si articola in quattro parti.

4.3.2.1 Adapter primari Ricevono uno stimolo esterno e lo traducono in una chiamata a un caso d'uso, senza contenere regole di business. Nella forma più comune sono i Controller HTTP. Il gestore dei task di workflow è un adapter primario equivalente, guidato da un messaggio SQS o da una callback di Step Functions invece che da una richiesta HTTP.

4.3.2.2 Dominio e applicazione Il dominio contiene le entità, le regole di business e le porte che definiscono le capacità del sistema, senza dipendenze da Eloquent, da un client AWS o da HTTP. L'applicazione implementa le porte primarie orchestrando il dominio esclusivamente attraverso le porte secondarie.

4.3.2.3 Adapter secondari Implementano la comunicazione con i sistemi esterni: il database PostgreSQL tramite repository Eloquent, i modelli AI tramite i wrapper di Bedrock e Textract, lo storage documentale tramite Flysystem e S3.

4.3.2.4 Esempio: il dominio Documents Un Controller riceve l'upload, lo valida e delega l'esecuzione al caso d'uso corrispondente, che salva il file e avvia la pipeline asincrona senza conoscere i dettagli di Step Functions. Il Worker raggiunge lo stesso dominio applicativo tramite un adapter primario diverso, guidato dal workflow, e delega ai casi d'uso di estrazione e revisione. Il risultato è persistito tramite il repository del dominio; un evento dedicato notifica audit e metriche. Il dominio Communications segue lo stesso schema.

4.4 Dettaglio dei Componenti

La tabella seguente quantifica la composizione interna dei due domini: casi d'uso, porte verso l'esterno, eventi di dominio e adapter primari.

Dominio	Casi d'uso	Porte secondarie	Eventi (= listener)	Adapter primario
Documents	9	6	13	6 Controller HTTP WorkflowTaskHandler +
Communications	12	6	17	5 Controller HTTP WorkflowTaskHandler +

Tabella 15: Metriche strutturali dei due domini esagonali

I due domini condividono un'unica porta secondaria, che astrae l'avvio di un'esecuzione Step Functions ed è implementata da un solo adapter (condiviso perché appartiene all'infrastruttura di orchestrazione, non ai domini di prodotto). La conformità alla Dependency Rule è verificata automaticamente: uno script, eseguito a ogni build della pipeline CI, analizza il codice delle due cartelle di dominio e interrompe la build in presenza di un riferimento a Illuminate, all'SDK AWS o a un modello Eloquent.

4.4.0.1 Documents

- **Ingresso:** UploadDocumentUseCase.
- **Orchestrazione pipeline:** StartDocumentWorkflowUseCase, RunOcrUseCase, ProcessDocumentUseCase, FinalizeDocumentWorkflowUseCase.
- **Consultazione e gestione:** ListDocumentsUseCase, DeleteDocumentUseCase.
- **Revisione umana:** ReviewDocumentUseCase.
- **Invio:** SendMessageUseCase.
- **Porte secondarie:** DocumentRepository (persistenza), OcrGatewayPort (Texttract), DocumentAiGatewayPort (Bedrock), DocumentStoragePort (S3/Flysystem), SendMessageRendererPort (rendering PDF del messaggio di invio), DocumentEventDispatcherPort (pubblicazione degli eventi di dominio).

4.4.0.2 Communications

- **Generazione:** GenerateCommunicationUseCase, GenerateCommunicationTextUseCase, GenerateCommunicationCoverUseCase.
- **Orchestrazione pipeline:** StartCommunicationWorkflowUseCase, FinalizeCommunicationUseCase.
- **Gestione della bozza:** CommunicationDraftUseCase, UpdateCommunicationCoverUseCase, DeleteCommunicationUseCase.
- **Consultazione, esportazione, valutazione:** ListCommunicationsUseCase, ExportCommunicationUseCase, RateCommunicationUseCase.
- **Configurazioni di prompt riutilizzabili:** PromptConfigurationUseCase.

- **Porte secondarie:** `CommunicationRepository` (persistenza), `CommunicationAiGatewayPort` (Bedrock), `CommunicationCoverStoragePort` (S3/Flysystem per le copertine), `CommunicationPdfRendererPort` (rendering PDF), `CommunicationEventDispatcherPort` (pubblicazione degli eventi di dominio), `PromptConfigurationRepository` (persistenza delle configurazioni salvate).

4.5 Architettura di Deployment

Il backend e la SPA frontend sono distribuiti come container separati, raggiunti da un unico entry point TLS. Una richiesta attraversa Traefik, l'edge (locale o CloudFront in produzione) e l'Nginx applicativo prima di raggiungere PHP-FPM, dove i Controller traducono la richiesta in una chiamata al caso d'uso corrispondente. Un secondo processo, il Worker, esegue lo stesso dominio applicativo raggiunto tramite i messaggi SQS e le callback di Step Functions invece che da HTTP: API e Worker sono due adapter primari sullo stesso nucleo applicativo, non due sistemi separati.

In locale l'intera topologia — orchestrazione Step Functions/SQS inclusa — è riprodotta con Docker Compose, LocalStack e Terraform, con gli stessi componenti usati in produzione.

4.6 Containerizzazione con Docker

L'intero stack è containerizzato; i dettagli di ciascun servizio sono descritti nella Sezione 2. I container principali sono:

- **Frontend (SPA):** esegue la build statica Angular, servita dall'edge; nessuna logica di business.
- **Backend (API):** esegue PHP-FPM dietro Nginx, gestisce le richieste HTTP verso il dominio applicativo.
- **Worker:** esegue lo stesso dominio applicativo del backend, in ascolto su Step Functions/SQS anziché su HTTP.
- **Database (PostgreSQL) e Cache (Redis):** persistenza applicativa e stato volatile, su volumi Docker dedicati.
- **Emulazione AWS (LocalStack) e provisioning (Terraform):** riproducono in locale, senza credenziali reali, l'infrastruttura di produzione.

4.7 Stile architetturale

Lo stile architetturale del backend è un monolite modulare organizzato per dominio: un nucleo esagonale limitato ai due domini di prodotto, circondato da un'infrastruttura condivisa che non vi è assoggettata. Tre caratteristiche lo definiscono:

- **Testabilità del dominio senza framework:** i casi d'uso di Documents e Communications sono istanziabili con `new`, senza container, database o LocalStack; la suite

di test dedicata usa implementazioni in memoria delle porte secondarie al posto di quelle reali.

- **Orchestrazione asincrona esternalizzata:** le operazioni onerose non transitano dal sistema di code nativo di Laravel, ma da Step Functions e SQS; l'adapter che le riceve è trattato allo stesso livello di un Controller HTTP.
- **Conformità verificata automaticamente:** la Dependency Rule è un controllo eseguito a ogni build, non una convenzione documentata.

Il perimetro dell'esagonale resta limitato per scelta esplicita: Identity, Audit, Observability, Support e l'infrastruttura condivisa di Workflow restano organizzati per servizio, perché nessuno di essi ha oggi — né è previsto abbia nell'arco della MVP — un'implementazione alternativa da rendere sostituibile.